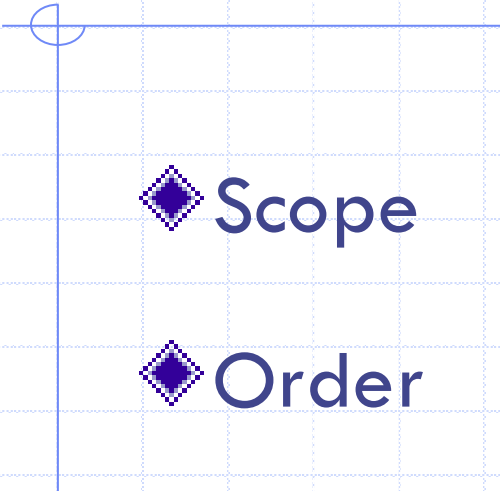


Standard Model of Programming Languages (SMoL)

CMSC 323

Examples are taken from tutorial created by the PLAI tutors

SMOL



- ◆ Scope
- ◆ Order
- ◆ Function as values
- ◆ Local bindings

Variable and Function Definitions

- ◆ A **block** is a sequence of definitions followed by a sequence of expressions. Blocks form tree-like structures
- ◆ Definitions might happen in top-level blocks or function bodies (which are also blocks)

Scope

The **primordial block** is a non-visible block enclosing the top-level block. This block defines values and functions (e.g., $+$ and $/$) that are provided by the language itself.

Scope

The **scope** of a variable is the region of a program where we can refer to the variable. Technically, it includes the block in which the variable is defined, and all sub-blocks of the block, **except** the sub-blocks (not including the block itself) in which the same name is re-defined. When the exception happens, that is, when the same name is defined in a sub-block, we say that the variable in the sub-block **shadows** the variable in the outer block

Scope

We say a variable reference (e.g., "the x in $(+ x 1)$ ") is **in the scope of** a declaration (e.g., "the x in $(\text{define } x 123)$ ") if and only if the former refers to the latter.

In English: The x used in $(+ x 1)$ is the one defined in $(\text{define } x 123)$

Scope

We find the declaration by looking in a series of blocks. We start with the block in which the variable reference occurs. If the variable is declared in the current block, we use that declaration. Otherwise, we look up the block in which the current block occurs, and so on recursively. (Specifically, if the current block is a function body, the next block will be the block in which the function definition is; if the current block is the top-level block, the next block will be the *primordial block*.) If the current block is already the primordial block and we still haven't found a corresponding declaration, the variable is **unbound**.

This method of scoping is known as **static scoping**

Scope

```
(define y 1)
(define (addy x)
  (+ x y))
(addy 2)
```

There is no `y` defined in `addy`'s block, so we look for `y` in the block where `addy` is defined – the top-level block

Result is 3

The value for `y` is gotten from the top-level environment where `y` is bound to 1

Scope

```
(define (addy x)
  (+ x y))
(define y 1)
(addy 2)
```

Result is 3

The value for *y* is gotten from the top-level environment where *y* is bound to 1

Note order is important here! Flipping last two lines results in error.

Scope

Defining `y` in both the top-level block and in the `addy`'s block

```
(define y 100)
(define (addy x)
  (local [(define y 200)]
    (+ x y)))
(addy 2)
```

Scope

```
(define y 100)
(define (addy x)
  (local [(define y 200)]
    (+ x y)))
(+ (addy 2) y)
```

(addy 2) is 202, as usual. (+ (addy 2) y) is in the top-level block, so it refers to the y in the top-level block. The result is the value of (+ 202 100), which is 302.

Scope

What if we do not have a top-level `y`?

```
(define (addx x)
  (local [(define y 200)]
    (+ x y)))
(+ (addx 2) y)
```

The last line of code is in the top-level block, so it refers to the `y` in the top-level block if there is one. No `y` is defined in the top-level block, so the program errors.

Scope

```
(define x 1)
(define (main)
  (local [(define x 2)
          (define (get-x) x)]
    (get-x)))
(main)
```

returns the value of `(get-x)`. The value of `(get-x)` is 2 because `get-x` is defined in `main`'s block, where `x` is 2.

Scope

```
(define x 1)
(define (get-x) x)

(define (main)
  (local [(define x 2)]
    (get-x)))
(main)
```

(main) returns the value of (get-x). The value of (get-x) is 1 because get-x is defined in the top-level block, where x is 1.

Scope

```
(define x 1)
(define (main)
  (local [(define x 2)]
    (get-x)))
(define (get-x) x)
(main)
```

(main) returns the value of (get-x). The value of (get-x) is 1 because get-x is defined in the top-level block, where x is 1.

Scope

What if both `x` and `get-x` are defined in `main`'s block?

```
(define x 1)
(define (main)
  (local [(define (get-x) x)
          (define x 2)]
    (get-x)))

(main)
```

`(main)` returns the value of `(get-x)`. The value of `(get-x)` is `2` because `get-x` is defined in the `main`'s block, where `x` is `2`.

Scope

It is an error to define a variable twice within a single block

```
(define x 1)
```

```
(define x 2)
```

x

Scope

It is an error to define a variable twice within a single block

```
(define (f x x)
```

```
  (+ x x))
```

```
(f 1 2)
```

Scope: Exercise

```
(define (main)
  (local [(define (getx) x)
          (define y (getx))
          (define x 10)]
    y))
(main)
```

x is not defined

Scope: Exercise

```
(define (main)
  (local [(define (getx) x)
          (define y (getx))
          (define x 10)]
    (getx)))
(main)
```

x is not defined

Scope: Exercise

```
(define (main)
  (local [(define (getx) x)
          (define x 10)]
    (getx)))
(main)
```

Scope: Exercise

```
(define (main)
  (local [(define (getx) x)
          (define x 10)]
    (getx)))
(main)
```

So the question: why?

Scope: Exercise

```
(define (main)
  (local [(define (getx) x)
          (define x 10)]
    (getx)))
(main)
```

So the question: why?

In Plait, local definitions within a local binding are evaluated sequentially at runtime, but all local function bodies are evaluated lazily when the function is actually called.

Scope: Exercise

```
(define x 5)
(define (set1 x y)
  (set! x y))
(define (set2 a y)
  (set! x y))
(set1 x 6)
x
(set2 x 7)
x
```

5 7

Order

Every definition of a variable evaluates the expression immediately and binds the variable to the value, even if the variable is not used later in the program. This is called **eager evaluation**.

Order

Every **function call** evaluates the actual parameters immediately and binds the values to formal parameters, even when the formal parameter is not used in the function.

Order

A sequence of *defines* binds the variables to the values in **top-to-bottom, left-to-right order**.

It is an error to evaluate a variable reference before binding it to a value.

Order

```
(define (getx) x)
```

```
(define y (getx))
```

```
(define x 12)
```

y

Unbound identifier

Exercise: Order

```
(define x 2)
(define (main)
  (local [(define (getx) x)
          (define y (getx))
          (define x 3)]
    y))
```

```
(main)
```

Unbound identifier

Mutations

- ◆ `set!` Mutates *existing* bindings
- ◆ `set!` expressions produce a special value, the *void* value, which indicates that evaluating the expression is not meant to produce an interesting value; rather, the evaluation is only for its "*side effects*".

Mutations

Side effect

A function is said to have a side effect if it has any observable effect other than the primary effect of returning a value

Examples:

- ◆ Changing the state of a variable
- ◆ Performing I/O

Mutations

- ◆ The variable (if any) mutated by set! is dictated by scope
- ◆ using set! to mutate a variable **won't mutate the binding of other variables**, no matter how the definition of the first variable is related to the second variable. e.g., given

(define y x)

Mutating x or y will not mutate the other

Exercise: Mutations

```
(define x 12)
```

```
(define y x)
```

```
(set! x 0)
```

x

y

0 12

Exercise: Mutations

```
(define x 12)
```

```
(define (f)
```

```
  (set! x 0))
```

```
(f)
```

x

0

Exercise: Mutations

```
(define x 1)
(define (getx) x)
(define (setx new-val)
  (set! x new-val))
(getx)
(setx 2)
(getx)
```

1 2

Exercise: Mutations

```
(define x 12)
(define (set-and-return y)
  (begin
    (set! y 0)
    x))
(set-and-return x)
```

12

Exercise: Mutations

What if we set x and return y?

```
(define x 12)
```

```
(define (set-and-return y)
```

```
  (begin
```

```
    (set! x 0)
```

```
    y))
```

```
(set-and-return x)
```

12 ^x
0

Exercise: Mutations

```
(define x 5)
(define (set1 x y)
  (set! x y))
(set1 x 6)
x
```

5

Functions as Values

Functions are values. Therefore, variables can be bound to functions and, functions can return functions

Functions as Values

- ◆ Some programming languages (e.g. Java) do not consider functions or methods a kind of value
- ◆ Other programming languages like Python and Javascript do
- ◆ Example:

```
(define (double x)
```

```
  (+ x x))
```

```
(define f double)
```

```
(f 2)
```

```
(f 3)
```


Functions as Values

Functions remember the environment that they refer to. Function values are called **closures** because function bodies are enclosed by the environment in which the function values are created.

Exercise: Functions as Values

What is the result of running this program?

```
(define (get) 42)
```

```
(define f get)
```

```
(define g f)
```

```
(g)
```

A function assigned to variables

Exercise: Functions as Values

What is the result of running this program?

```
(define (twice f x)  
  (f (f x)))
```

```
(define (double x)  
  (+ x x))
```

```
(twice double 1)
```

A function passed as a parameter

Exercise: Functions as Values

What is the result of running this program?

```
(define (make-getter)
  (local [(define x 1)
          (define (getx) x)]
    getx))

(define getx (make-getter))

(getx)
```

Exercise: Functions as Values

What is the result of running this program?

```
(define x 1)
(define (make-f)
  (local [(define (addx y) (+ x y))]
    addx))
(define f (make-f))
(set! x 2)
(f x)
```

Functions as Values

What is the result of running this program?

```
(define (make-counter count)
  (local [(define (counter)
              (begin
                (set! count (+ count 1))
                count))])
    counter))
```

```
(define f (make-counter 0))
(define g (make-counter 0))
```

```
(f)
(f)
(g)
```

Functions as Values

```
(define (f x)
  (lambda (y) (+ x y)))
(define x 0)
((f 2) 1)
```

Functions as Values

```
(define x 1)
```

```
(define (f y)
```

```
  (+ x y))
```

```
(set! x 2)
```

```
(f x)
```


Functions as Values

`(define (f x ...) body)`

Is shorthand for

`(define f (lambda (x ...) body))`

Local Binding

- ◆ A **let** expression binds its left-hand-side variables such that its body is in the scope of all the variables
- ◆ In a let, the scope of a left-hand-side variable includes none of the right-hand-side expressions.

Exercise: Local Binding

What is the result of running this program?

```
(let ([x 2])  
  (let ([y 30])  
    (+ x y)))
```

Exercise: Local Binding

What is the result of running this program?

```
(define x 3)
```

```
(let ([x 5])
```

```
  (+ x 1))
```

```
(+ x 2)
```

Exercise: Local Binding

What is the result of running this program?

```
(let ([x 2]  
      [y (+ x 1)]])  
  (+ x y))
```

Exercise: Local Binding

What is the result of running this program?

```
(let ([x 123]  
      [getx (lambda () x)])  
  (getx))
```

Exercise: Local Binding

What is the result of running this program?

```
(let ([f (lambda (n)
            (or (equal? n 0)
                (f (- n 1)))]])
  (f 3))
```

Exercise: Local Binding

```
(let ([x 2]  
      [y 3])  
  (+ x y))
```


Exercise: Local Binding

```
(let ([x 2]  
      [y x])  
  (+ x y))
```

Exercise: Local Binding

```
(let ([x 2])  
  (let ([y x])  
    (+ x y)))
```

